Single-Cell RNA-seq Analysis of Drosophila Ovarian Follicle Cells

BIOENG-420 – Single-cell Biology

March 18, 2026

Group 07 - Ovary

Name	SCIPER
Clotilde Christine Noelle Bévoz	346441
Johann Clausen	346400
Nafi Anna Dramé	340970

Link to the repository: source code

1 Introduction

1.1 Project Objective

The objective of this project is to analyze a heterogeneous population of *Drosophila* cells originating from multiple tissues in order to identify distinct **cell types**, determine their associated **marker genes**, and infer their **probable tissue origins**. The analysis is based on a single-cell RNA sequencing dataset and will involve standard preprocessing, dimensionality reduction, clustering, and marker gene identification in order to characterize transcriptionally distinct cell populations.

1.2 Dataset

The dataset used in this analysis is stored in the file `data/ovary.tsv`. It contains **single-cell RNA sequencing count data obtained from Drosophila ovary cells**. The data are organized in a tabular format where rows correspond to genes and columns correspond to individual cells. Each value represents the number of sequencing counts detected for a given gene in a given cell.

Since this dataset represents a heterogeneous population of cells originating from the **Drosophila ovary**, it includes multiple cell types potentially involved in ovarian structure and function.

1.3 Computational Tools

The analysis will be performed in **Python**, using several widely adopted libraries for single-cell analysis and data visualization:

- **Scanpy** – used for preprocessing, dimensionality reduction, clustering, and visualization of single-cell data.
- **Pandas** – used for data loading and manipulation.
- **Matplotlib** and **Seaborn** – used for producing visualizations and exploratory plots.

- **DRscDB** and **FlyBase** and relevant scientific literature – used to interpret marker genes and infer likely cell-type identities.

2 Analysis Workflow

Step	Section	Summary
1	Data Loading and Quality Control	Load the TSV file into a Scanpy AnnData object, inspect genes per cell and cells per gene, and filter low-quality or uninformative cells/genes.
2	Normalization and Preprocessing	Normalize counts per cell, apply log transformation, identify highly variable genes, scale data, and run PCA to retain major variation.
3	Dimensionality Reduction	Build a neighborhood graph from PCA and compute UMAP and/or t-SNE to visualize transcriptional heterogeneity in low-dimensional space.
4	Clustering	Apply graph-based clustering (for example Leiden) to identify transcriptionally similar groups and visualize cluster separation on UMAP/t-SNE.
5	Marker Gene Identification	Perform differential expression to find genes enriched in each cluster and define marker genes for molecular identity.
6	Cell-Type Annotation	Compare marker genes with FlyBase and literature to assign tentative cell-type identities and likely tissue origins for each cluster.

2.1 Expected Outcomes

At the end of this analysis, the dataset should reveal several transcriptionally distinct clusters corresponding to candidate cell populations present in the *Drosophila* ovary. For each identified cluster, differential expression analysis will allow the identification of **marker genes** that characterize the molecular identity of the cells within that group. These marker genes can then be compared with existing annotations and published gene expression profiles in databases such as FlyBase and in the scientific literature. Through this comparison, it will be possible to propose **tentative cell-type**

annotations and infer the likely biological identity of each cluster. Overall, the analysis should provide an overview of the **cellular diversity present in the dataset**, including the number of detected cell populations and their probable tissue or functional origins within the ovary.

During preprocessing, it is important to ensure that **gene names are unique**, as duplicate identifiers can lead to errors during downstream analysis steps.

3 1. Data Loading and Quality Control

[1]:
```
from __future__ import annotations

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from adjustText import adjust_text

import scanpy as sc
```

3.0.1 a) Inspecting the file structure and loading of the data:

The first step consists of loading the data from the ovary.tsv file and visualize the structure of the first few rows (let's say the first five ones) to understand its structure for further analysis.

[2]:
```
import os

print("\nSearching for ovary.tsv ...")
for root, dirs, files in os.walk("."):
    for file in files:
        if file == "ovary.tsv":
            ovary_path = os.path.join(root, file)
            print(f"Found: {ovary_path}")
            break
```

```
Searching for ovary.tsv …
Found: ./data/ovary.tsv
```

[3]:
```
# Read just the first few rows to understand the file structure
df_peek = pd.read_csv(ovary_path, sep="\t", nrows=10, index_col=0, header=None)
df_peek.index.name = "genes"
df_peek.columns = [f"cell_{i}" for i in range(1, df_peek.shape[1] + 1)]
df_peek.columns.name = "cells"

print(f"Shape of preview: {df_peek.shape[1]} cells x {df_peek.shape[0]} genes")
print("Genes:", df_peek.index.tolist())
df_peek.head(10)
```

```
Shape of preview: 7549 cells x 10 genes
Genes: ['128up', '14-3-3epsilon', '14-3-3zeta', '140up', '18SrRNA-Psi:CR41602',
```

```
'18w', '26-29-p', '28SrRNA-Psi:CR40596', '28SrRNA-Psi:CR40741', '28SrRNA-
Psi:CR41609']
```

[3]:

cells	cell_1	cell_2	cell_3	cell_4	cell_5	cell_6	cell_7	\
genes								
128up	0	0	0	0	0	0	0	
14-3-3epsilon	1	0	2	2	1	1	2	
14-3-3zeta	1	2	1	2	1	0	3	
140up	0	0	1	0	0	0	0	
18SrRNA-Psi:CR41602	0	0	0	0	0	0	0	
18w	0	0	0	0	0	0	0	
26-29-p	0	1	4	1	0	1	6	
28SrRNA-Psi:CR40596	0	0	1	0	0	0	0	
28SrRNA-Psi:CR40741	0	0	0	0	0	0	0	
28SrRNA-Psi:CR41609	0	0	0	0	0	0	0	

cells	cell_8	cell_9	cell_10	…	cell_7540	cell_7541	\
genes				…			
128up	0	0	0	…	0	0	
14-3-3epsilon	0	3	0	…	0	4	
14-3-3zeta	0	3	8	…	2	5	
140up	0	0	0	…	0	0	
18SrRNA-Psi:CR41602	0	0	0	…	0	0	
18w	0	0	1	…	0	0	
26-29-p	2	3	1	…	0	0	
28SrRNA-Psi:CR40596	1	0	1	…	0	0	
28SrRNA-Psi:CR40741	0	0	0	…	0	0	
28SrRNA-Psi:CR41609	0	0	0	…	0	0	

cells	cell_7542	cell_7543	cell_7544	cell_7545	cell_7546	\
genes						
128up	0	0	0	0	0	
14-3-3epsilon	1	0	0	0	1	
14-3-3zeta	3	3	1	1	0	
140up	0	0	0	0	0	
18SrRNA-Psi:CR41602	0	0	1	0	0	
18w	0	0	0	0	0	
26-29-p	2	0	0	0	0	
28SrRNA-Psi:CR40596	0	0	0	0	0	
28SrRNA-Psi:CR40741	0	0	0	0	0	
28SrRNA-Psi:CR41609	0	0	0	0	0	

cells	cell_7547	cell_7548	cell_7549
genes			
128up	0	0	0
14-3-3epsilon	3	4	0
14-3-3zeta	2	8	6

```
140up                        0          0          0
18SrRNA-Psi:CR41602          0          0          0
18w                          0          0          0
26-29-p                      2          0          0
28SrRNA-Psi:CR40596          0          0          0
28SrRNA-Psi:CR40741          0          0          0
28SrRNA-Psi:CR41609          0          0          0

[10 rows x 7549 columns]
```

The `ovary.tsv` file is organized as a **gene expression count matrix**.

Row 1 corresponds to a **cell type label (`128up`) followed by expression values** for that cell type across many genes. **Row 2 onward** correspond to **individual genes**, where: - The **first element of the row is the gene name** (e.g., `14-3-3epsilon`). - The remaining values are **expression counts** of that gene across many cells.

The file follows this structure:

cells / genes	cell_1	cell_2	cell_3	...
128up	0	0	0	...
14-3-3epsilon	1	0	2	...
14-3-3zeta	...	...	...	...

Then, we can pursue furhter analysis taking into account that the **first column contains identifiers** (either the cell-type label in the first row or gene names in subsequent rows); the **remaining columns contain numeric expression counts**. Each value represents **how many transcripts of a gene were detected in a specific cell**.

This structure must be interpreted correctly when loading the file so that **gene names become the index and cells become columns for single-cell analysis**.

Then, we can load the data set and prepare it for **single-cell RNA analysis**.

[4]:
```
# Load TSV with gene names as index
df = pd.read_csv(ovary_path, sep="\t", index_col=0, header=None)
df.index.name = "genes"
df.columns = [f"cell_{i}" for i in range(1, df.shape[1] + 1)]
df.columns.name = "cells"

# Convert data values to numeric (index is already string gene names, safe)
df = df.apply(pd.to_numeric, errors='coerce').fillna(0)
```

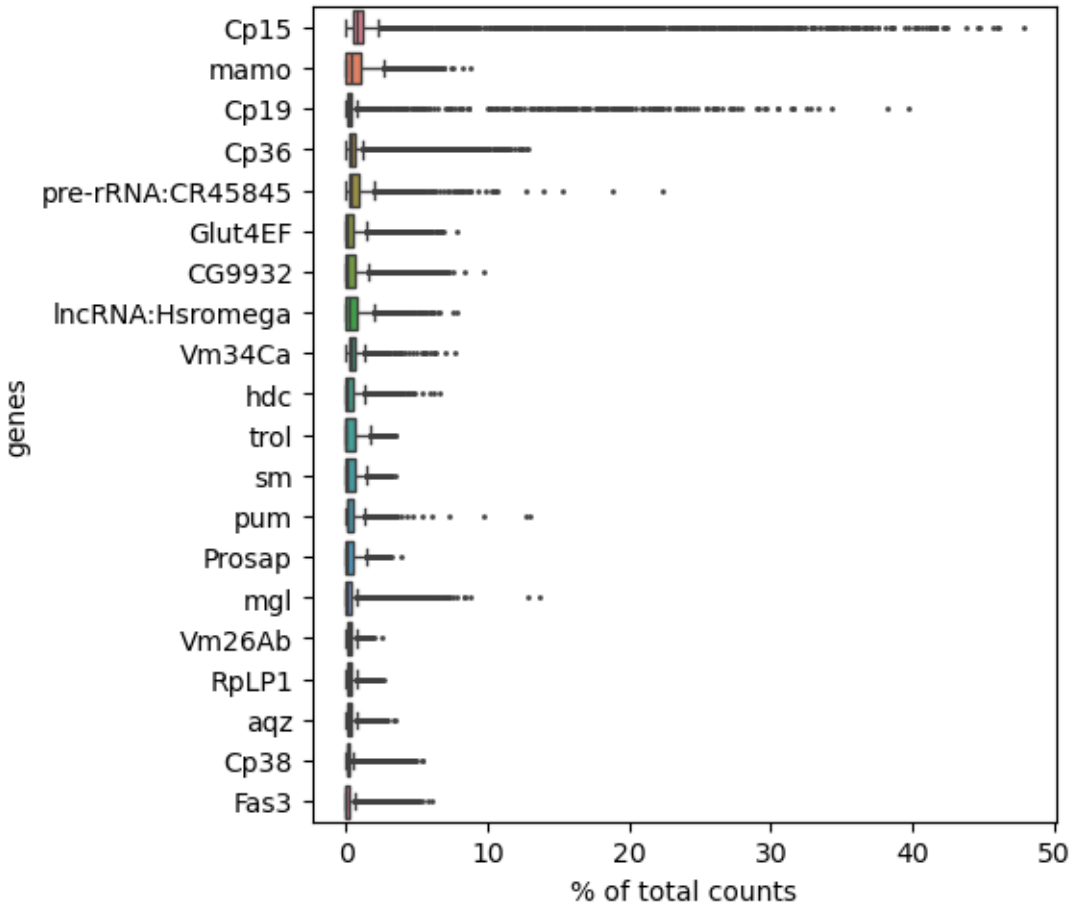
[5]:
```
# Create AnnData
adata = sc.AnnData(df.T)  # transpose: cells x genes

print("adata.var_names (genes):", adata.var_names[:10].tolist())
print("adata.obs_names (cells):", adata.obs_names[:5].tolist())
```

```
adata.var_names_make_unique()

# Filter
sc.pp.filter_genes(adata, min_cells=3)
sc.pp.filter_cells(adata, min_genes=200)

print(f"\nShape before filtering: {df.shape[1]} cells x {df.shape[0]} genes")
print(f"Shape after filtering: {adata.shape[0]} cells x {adata.shape[1]} genes")

# Check if known markers are findable
print("\nChecking for known markers in adata.var_names:")
markers = ['nos', 'bam', 'tj', 'eya', 'fas3']
for gene in markers:
    matches = [g for g in adata.var_names if gene.lower() in g.lower()]
    print(f"{gene} -> {matches[:3]}")
```

```
adata.var_names (genes): ['128up', '14-3-3epsilon', '14-3-3zeta', '140up',
'18SrRNA-Psi:CR41602', '18w', '26-29-p', '28SrRNA-Psi:CR40596', '28SrRNA-
Psi:CR40741', '28SrRNA-Psi:CR41609']
adata.obs_names (cells): ['cell_1', 'cell_2', 'cell_3', 'cell_4', 'cell_5']

Shape before filtering: 7549 cells x 11944 genes
Shape after filtering: 7549 cells x 11218 genes

Checking for known markers in adata.var_names:
nos -> ['Inos', 'Nos', 'Nost']
bam -> ['bam']
tj -> ['Patj', 'stj', 'tj']
eya -> ['eya']
fas3 -> ['Fas3']
```

This steps enables the loading of the gene expression `ovary.tsv` file, converts all values to numeric, and creates a Scanpy `AnnData` object with cells as rows and genes as columns. It ensures gene names are unique, filters out low-quality genes and cells, and checks for the presence of key marker genes, preparing the dataset for downstream **single-cell analysis**.

[6]:
```
sc.pl.highest_expr_genes(adata, n_top=20)
```



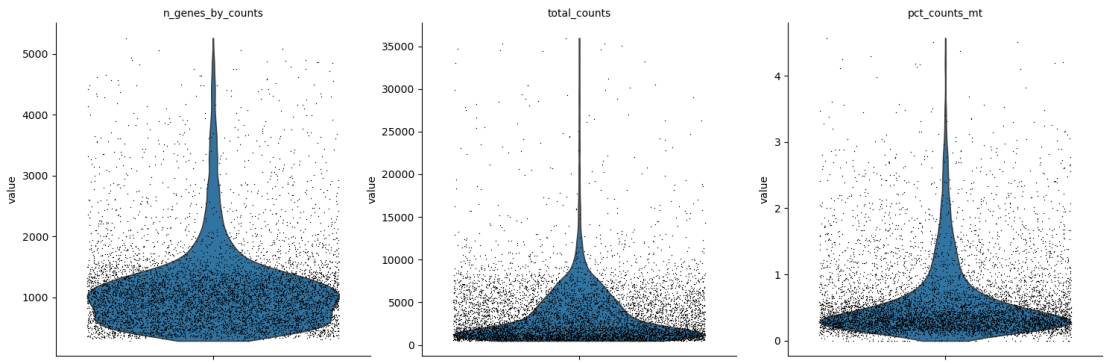
3.0.2 b) Quality control:

After creating the `AnnData` object, the next step is to **assess cell quality in the dataset**. This quality control aims to remove low-quality or abnormal cells that could bias downstream analyses. Three metrics are calculated for each cell: - First, **the number of genes detected per cell** (`n_genes`) is recorded. Cells with very few detected genes are likely dead or poorly sequenced, whereas cells with unusually high gene counts may represent doublets, in which two cells were captured as one. - Second, **the total counts per cell** (`n_counts`) are measured to evaluate sequencing depth and to identify cells with abnormally low or high transcript abundance. - Third, **the percentage of mitochondrial transcripts** (`percent_mito`) is calculated as the fraction of a cell's transcripts derived from mitochondrial genes. Elevated mitochondrial content often indicates stressed or dying cells, and in fly ovary data, these genes are typically prefixed with `mt:`.

To **examine the distributions of these metrics and identify outliers**, *violin plots* are used, providing a visual representation of variation across all cells. Based on these visualizations, cells with extreme values are removed, including those with very low or very high gene counts or with excessively high mitochondrial transcript percentages. This filtering step ensures that downstream analyses, such as clustering and differential expression, are performed on high-quality cells that reflect biologically meaningful variation.

[7]:
```
# annotate the group of mitochondrial genes as "mt"
adata.var["mt"] = adata.var_names.str.startswith("mt:")
sc.pp.calculate_qc_metrics(adata, qc_vars=["mt"], percent_top=None,␣
↪log1p=False, inplace=True)
```

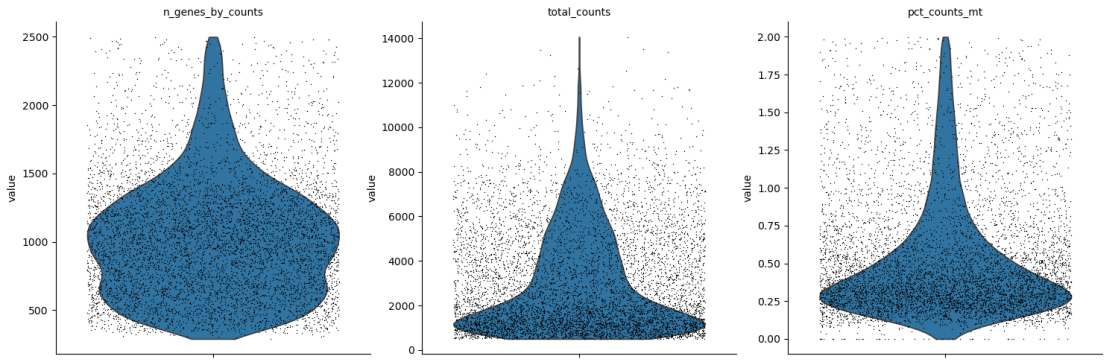
[8]:
```
sc.pl.violin(
    adata,
    ["n_genes_by_counts", "total_counts", "pct_counts_mt"],
    jitter=0.4,
    multi_panel=True,
)
```



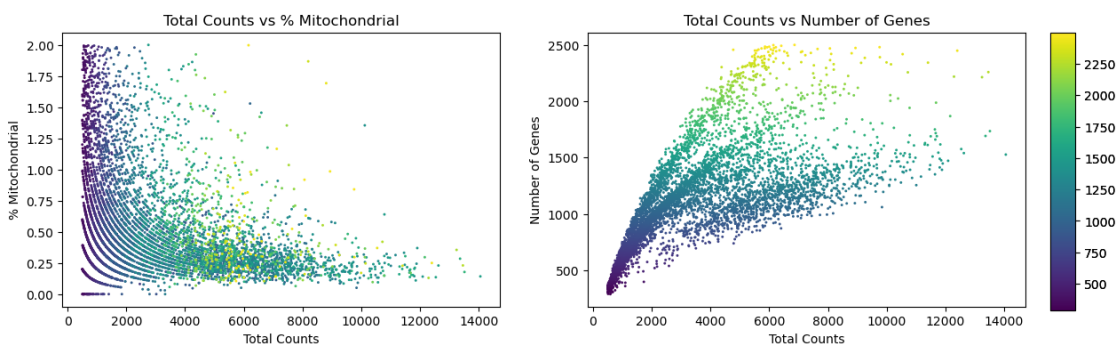
By observing the violin plots, we can remove cells that have too many mitochondrial genes expressed or too many total counts, using the following thresholds: - **Total counts**: 200 < total counts < 2500 - **Mitochondrial gene percentage**: pct_counts_mt < 2%

[9]:
```
adata = adata[
    (adata.obs.n_genes_by_counts < 2500) & (adata.obs.n_genes_by_counts > 200)␣
↪& (adata.obs.pct_counts_mt < 2),
    :,
].copy()
adata.layers["counts"] = adata.X.copy()
```

[10]:
```
sc.pl.violin(
    adata,
    ["n_genes_by_counts", "total_counts", "pct_counts_mt"],
    jitter=0.4,
    multi_panel=True,
)
```



[11]:
```
fig, axs = plt.subplots(1, 2, figsize=(14, 4))
sc.pl.scatter(adata, x="total_counts", y="pct_counts_mt",␣
 ↪color="n_genes_by_counts", show=False, ax=axs[0])
sc.pl.scatter(adata, x="total_counts", y="n_genes_by_counts",␣
 ↪color="n_genes_by_counts", show=False, ax=axs[1])
box = (0.92, 0.10, 0.02, 0.78)
fig.axes[-1].set_position(box)  # move colorbar to the right
fig.axes[-2].set_position(box)  # move colorbar to the right
axs[0].set_title("Total Counts vs % Mitochondrial")
axs[1].set_title("Total Counts vs Number of Genes")
axs[0].set_xlabel("Total Counts")
axs[0].set_ylabel("% Mitochondrial")
axs[1].set_xlabel("Total Counts")
axs[1].set_ylabel("Number of Genes")
plt.show()
```



4 2. Normalization and Preprocessing

4.0.1 a) Normalization:

After quality control, the next step is **normalization**.

In scRNA-seq, the amount of RNA captured per cell can vary greatly. To make gene expression values comparable across cells, we normalize each cell's counts:

1. Scale each cell's total counts to a constant (default: 10,000).

2. Log-transform the scaled counts to reduce skewness and make the data more suitable for downstream analysis.

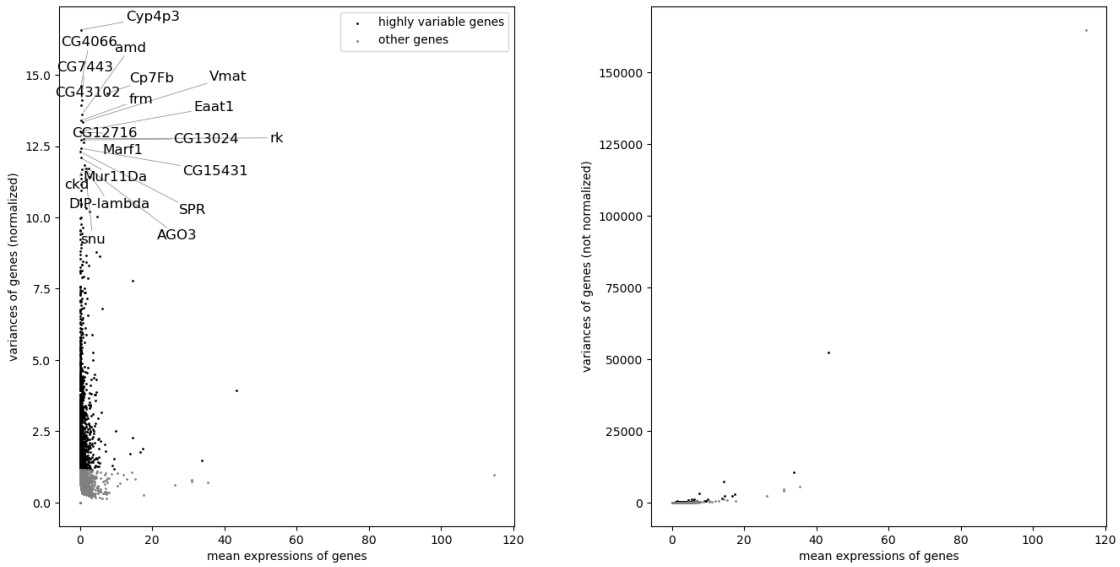
This is analogous to TPM (Transcripts Per Million) normalization used in bulk RNA-seq.

[12]:
```
# Normalize the data (total-count normalize to 10,000 counts per cell and␣
↪log-transform)
sc.pp.normalize_total(adata, target_sum=1e4)
sc.pp.log1p(adata)  # log(x + 1) transform

# Save log-normalized (unscaled) matrix for DE
adata.raw = adata.copy()
```

4.0.2 b) Feature selection for following heterogeneity analysis: identifying highly variable genes

[13]:
```
sc.pp.highly_variable_genes(
    adata,
    layer="counts",
    n_top_genes=2000,
    min_mean=0.0125,
    max_mean=3,
    min_disp=0.5,
    flavor="seurat_v3",
)
```

[14]:
```
sc.pl.highly_variable_genes(adata, show=False)

fig = plt.gcf()
ax = fig.axes[0]  # main scatter axis

hvg = adata.var[adata.var["highly_variable"]]
top_features = hvg.sort_values("variances_norm", ascending=False).head(20).index

texts = []
for gene in top_features:
    x = adata.var.loc[gene, "means"]
    y = adata.var.loc[gene, "variances_norm"]
    texts.append(ax.text(x, y, gene, fontsize=12))

adjust_text(texts, ax=ax, arrowprops=dict(arrowstyle="-", color="gray", lw=0.5))
fig.set_size_inches(16, 8)
plt.show()
```



In order to explore cellular heterogeneity, we performed **feature selection** by identifying highly variable genes (HVGs) across all cells. Genes that are either lowly expressed or exhibit similar expression across cells provide limited information and can obscure differences between distinct cell populations. Therefore, selecting genes with the most variable expression enhances the detection of meaningful cell groups. Using a method analogous to Seurat's `FindVariableFeatures`, the top 3,000 highly variable genes were identified based on their standardized variance. A scatter plot of mean expression versus normalized dispersion was generated, and the 20 genes with the highest variability were highlighted. This step ensures that subsequent analyses, such as clustering and dimensionality reduction, focus on the most informative features for capturing cellular heterogeneity.

4.0.3 c) Data scaling:

Because genes vary in their baseline expression levels and distributions, their raw contributions to downstream analyses would be unequal, with highly expressed genes dominating the results. To prevent this bias, the data are **scaled** across the selected features, ensuring that each gene contributes comparably to the analysis, a standard practice in data preprocessing across many fields.

[15]:
```
adata.layers["scaled"] = adata.X
sc.pp.regress_out(adata, ["total_counts", "pct_counts_mt"], layer="scaled")
sc.pp.scale(adata, max_value=10, layer="scaled")
```

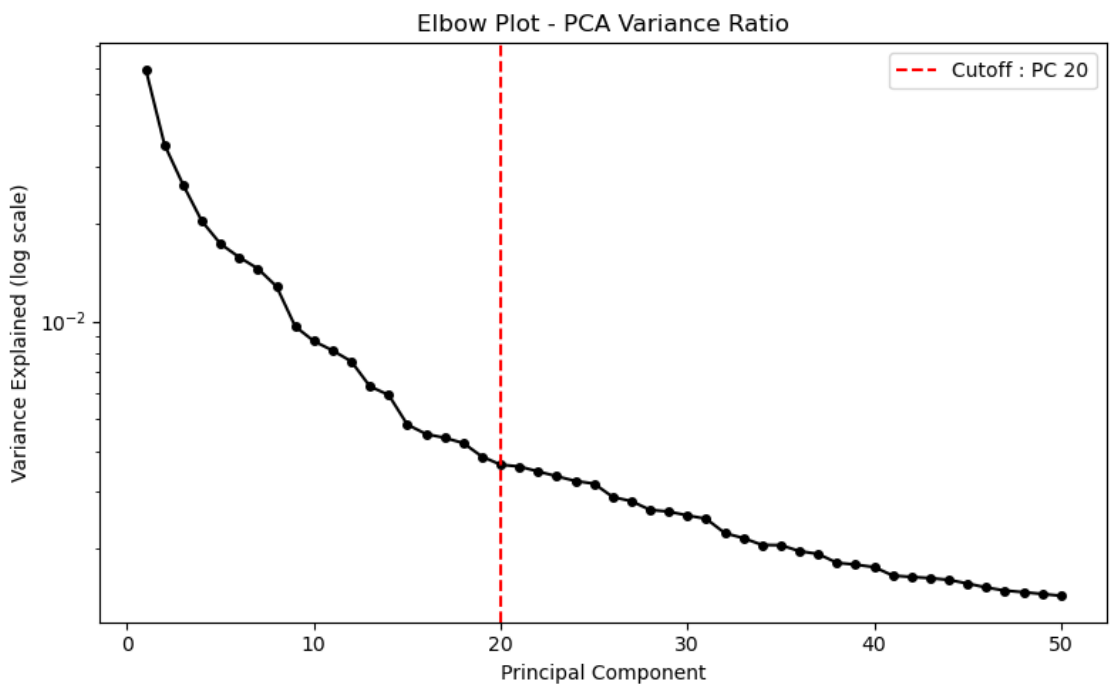
5 3. Dimensionality Reduction

After selecting highly variable genes and scaling the data, we performed **linear dimensionality reduction** using principal component analysis (PCA). PCA transforms the high-dimensional gene expression data into a smaller set of uncorrelated components, capturing the major sources of variation among cells. This step reduces computational load, enhances signal robustness in the

inherently sparse scRNA-seq data, and facilitates downstream analyses such as clustering and visualization.

We calculated the principal components using the top highly variable genes and inspected the variance explained by each component. An **elbow plot** of the explained variance was generated to determine the number of informative PCs, with the first 20 components identified as capturing the majority of biologically meaningful variation.

[16]:
```
sc.tl.pca(adata, svd_solver='arpack')

variance_ratio = adata.uns['pca']['variance_ratio']

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(range(1, len(variance_ratio) + 1), variance_ratio, 'o-', markersize=4,␣
 ↪color='black')
ax.axvline(x=20, color='red', linestyle='--', label='Cutoff : PC 20')
ax.set_yscale('log')
ax.set_xlabel('Principal Component')
ax.set_ylabel('Variance Explained (log scale)')
ax.set_title('Elbow Plot - PCA Variance Ratio')
ax.legend()
plt.tight_layout()
plt.show()
```

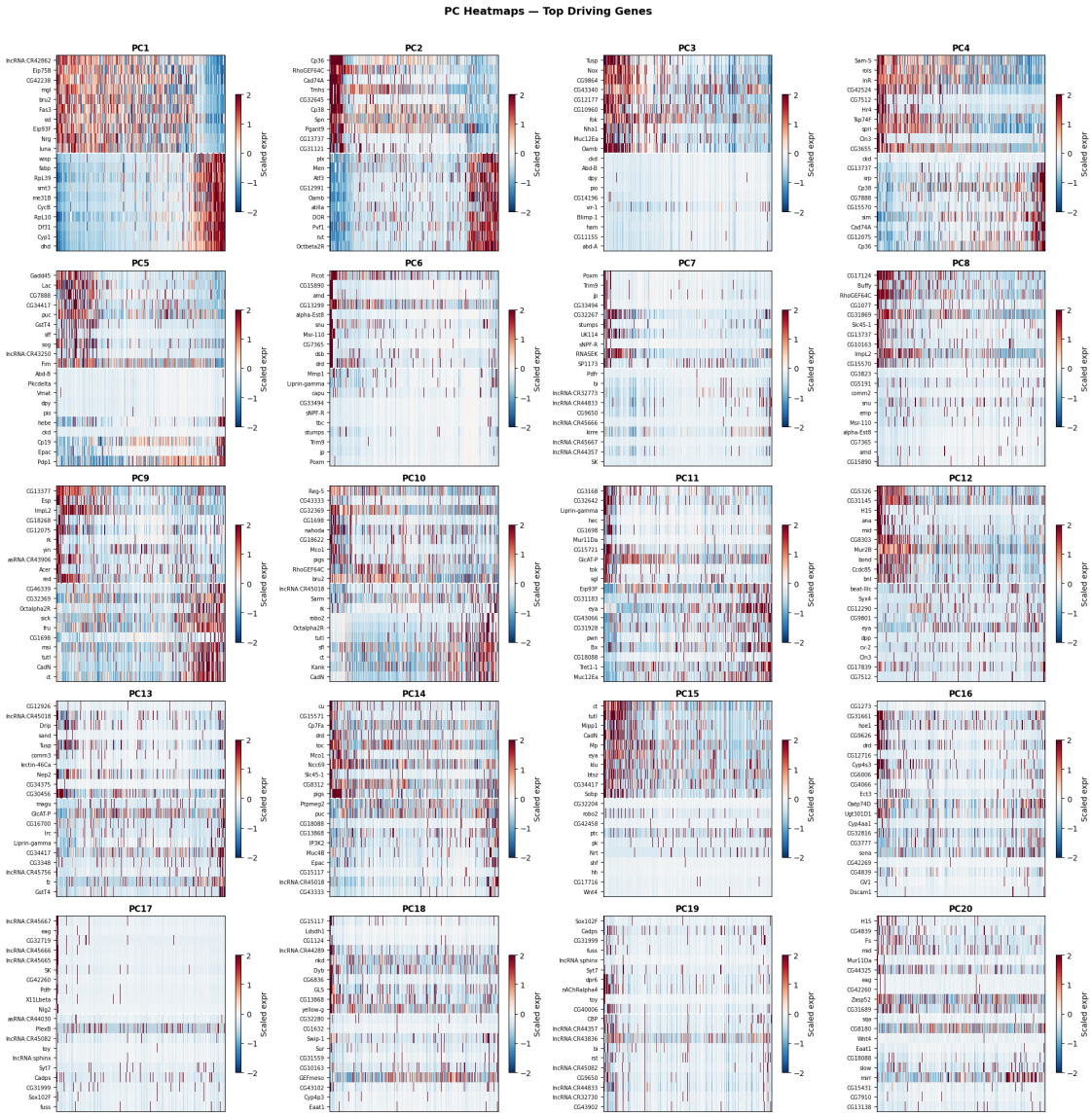


The elbow plot of the PCA variance ratio shows a **steep drop** for the first few principal components (PCs 1–5), indicating that these components capture the strongest sources of biological variation in

the dataset, likely reflecting major differences between cell types. Following this, there is a **gradual flattening** for PCs 5–20, which still represent meaningful but subtler biological signals, such as distinctions between cell subtypes or transient cellular states. Beyond PC 20, the curve forms a **flat tail**, suggesting that additional components mainly capture technical noise or stochastic cell-to-cell variability and are less informative for downstream analysis.

[17]:
```
def plot_pc_heatmap(adata, n_pcs=20, n_genes=10, n_cells=500):
    loadings = adata.varm['PCs']
    embeddings = adata.obsm['X_pca']

    ncols = 4
    nrows = int(np.ceil(n_pcs / ncols))
    fig, axes = plt.subplots(nrows, ncols, figsize=(ncols * 5, nrows * 4))
    axes = axes.flatten()

    for pc_idx in range(n_pcs):
        ax = axes[pc_idx]
        pc_loadings = loadings[:, pc_idx]

        # Balanced top/bottom genes
        top_pos = np.argsort(pc_loadings)[-(n_genes):]
        top_neg = np.argsort(pc_loadings)[:n_genes]
        gene_idx = np.concatenate([top_neg, top_pos])
        gene_names = adata.var_names[gene_idx]

        # Sort cells by PC score
        cell_order = np.argsort(embeddings[:, pc_idx])
        step = max(1, len(cell_order) // n_cells)
        cell_subset = cell_order[::step][:n_cells]

        # Extract expression
        if hasattr(adata.X, 'toarray'):
            expr = adata.X[cell_subset][:, gene_idx].toarray()
        else:
            expr = adata.X[cell_subset][:, gene_idx]

        # Plot
        im = ax.imshow(expr.T, aspect='auto', cmap='RdBu_r',
                       vmin=-2, vmax=2, interpolation='nearest')

        ax.set_title(f'PC{pc_idx + 1}', fontsize=11, fontweight='bold')
        ax.set_yticks(range(len(gene_names)))
        ax.set_yticklabels(gene_names, fontsize=7)
        ax.set_xticks([])  # hide cell numbers - not informative

        # Divider line between neg and pos genes
```

```
        ax.axhline(y=n_genes - 0.5, color='white', linewidth=1.5,␣
↪linestyle='--')

        plt.colorbar(im, ax=ax, shrink=0.6, label='Scaled expr')

    # Hide unused subplots
    for i in range(n_pcs, len(axes)):
        axes[i].set_visible(False)

    plt.suptitle('PC Heatmaps - Top Driving Genes', fontsize=14,␣
↪fontweight='bold', y=1.01)
    plt.tight_layout()
    plt.savefig('results/pc_heatmap_clean.png', dpi=150, bbox_inches='tight')
    plt.show()

plot_pc_heatmap(adata, n_pcs=20, n_genes=10, n_cells=500)
```

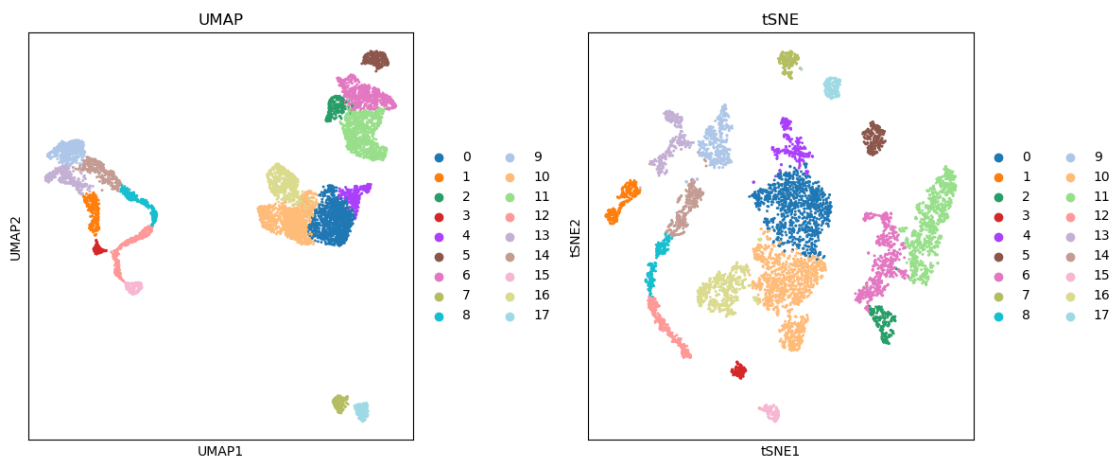


6 4. Clustering and Visualization

After performing linear dimensionality reduction with PCA, we further applied non-linear dimension reduction techniques to visualize the cellular heterogeneity in two dimensions.

[18]:
```
# Compute neighborhood graph (required before UMAP/tSNE)
# n_pcs match our elbow plot decision : 20
sc.pp.neighbors(adata, n_pcs=20)

# Run UMAP
sc.tl.umap(adata)
```

```
# Run t-SNE
sc.tl.tsne(adata, n_pcs=20)

# Run Leiden clustering
sc.tl.leiden(adata, resolution=0.5, flavor='igraph')  # adjust resolution to␣
 ↪get more/fewer clusters

# Visualize side by side
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
sc.pl.umap(adata, ax=axes[0], color ='leiden', show=False, title='UMAP')
sc.pl.tsne(adata, ax=axes[1], color ='leiden', show=False, title='tSNE')
plt.tight_layout()
plt.show()
```



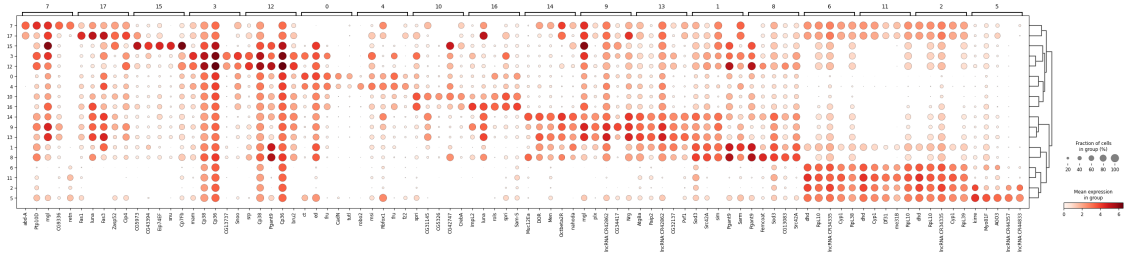
While PCA preserves the linear structure of the data and allows straightforward interpretation of principal components, it requires many dimensions to capture most of the information, making it unsuitable for direct 2D visualization. To overcome this limitation, we computed both **UMAP** (Uniform Manifold Approximation and Projection) and **t-SNE** (t-distributed Stochastic Neighbor Embedding) embeddings using the top 20 principal components as input, following the elbow plot analysis. These non-linear methods aim to **represent neighborhood relationships between cells** in a low-dimensional space, allowing clusters or continuum structures to be more easily observed. UMAP tends to preserve trajectory-like structures and continuous transitions between cell states, while t-SNE often provides clearer separation when discrete cell groups exist. Both embeddings were visualized side by side, and clustering with the *Leiden algorithm* was performed to **identify putative cell populations**, which can then be explored further using known marker genes.

7 5. Marker Gene Identification

To identify marker genes for each cluster, we performed **differential expression analysis** comparing each cluster against all other cells. This approach identifies genes that are significantly

upregulated in a specific cluster, serving as potential markers that characterize the molecular identity of the cells within that group. The top 100 marker genes for each cluster were extracted based on statistical significance and fold change. These marker genes can then be compared with known gene expression profiles from databases such as FlyBase and relevant scientific literature to propose tentative cell-type annotations and infer the likely biological identity of each cluster.

[19]:
```
# Compute DE and also store expression prevalence (fraction of non-zero cells)
sc.tl.rank_genes_groups(
    adata,
    'leiden',
    mask_var='highly_variable',
    method='t-test',
    pts=True,
 )
sc.tl.dendrogram(adata, groupby="leiden")
sc.pl.rank_genes_groups_dotplot(adata, n_genes=5, groupby="leiden", show=True)
```



The clustering along the diagonal is consistent with the presence of distinct cell populations, while the off-diagonal patterns may indicate shared gene expression profiles or transitional states between clusters.

[20]:
```
result = adata.uns["rank_genes_groups"]
available_clusters = list(result["names"].dtype.names)

# First 3 clusters
clusters_to_show = available_clusters[:3]
min_pct = 0.10

tables = []
for cl in clusters_to_show:
    # Pull tidy DE table for one cluster
    df_cl = sc.get.rank_genes_groups_df(adata, group=cl).copy()

    # Keep genes expressed in >=10% of cells in group OR reference
    if {"pct_nz_group", "pct_nz_reference"}.issubset(df_cl.columns):
        df_cl = df_cl[
            (df_cl["pct_nz_group"] >= min_pct)
```

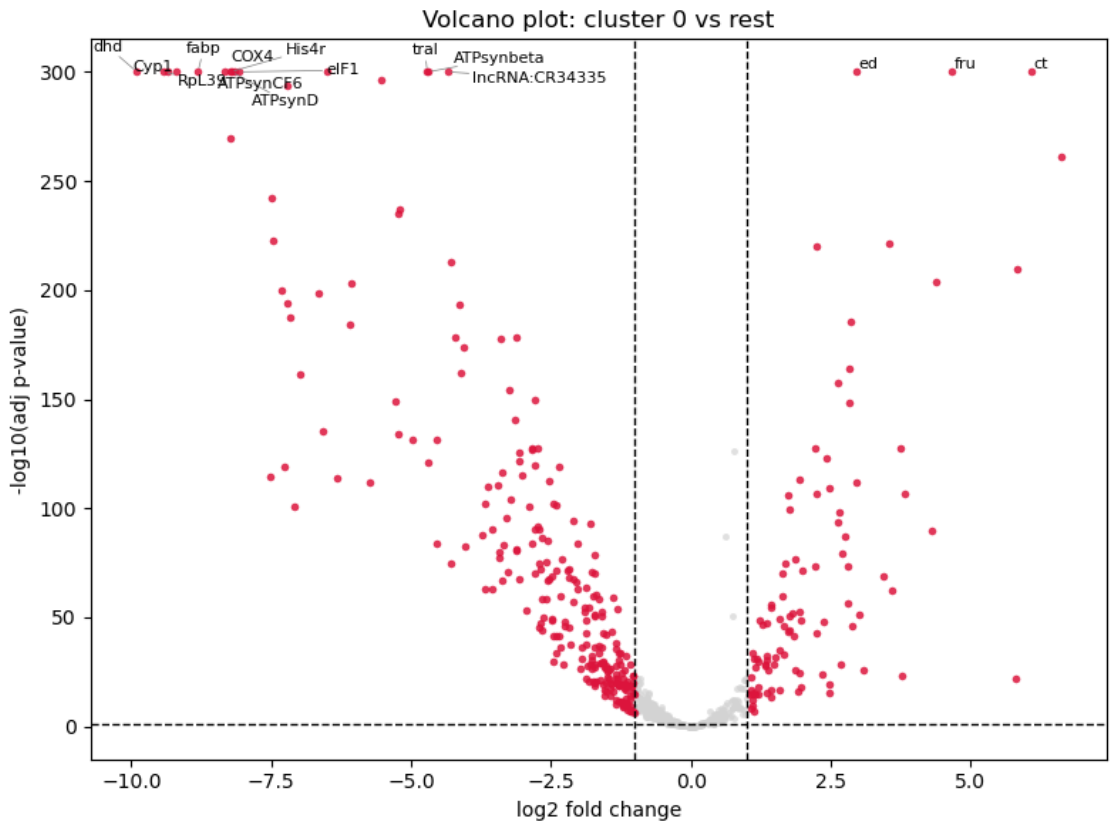
```
            | (df_cl["pct_nz_reference"] >= min_pct)
        ]

    df_cl = df_cl.rename(columns={
        "names": "gene",
        "logfoldchanges": "log2FoldChange",
        "pvals": "pval",
        "pvals_adj": "pval_adj",
        "scores": "score",
    })

    # Keep only upregulated genes (genuine markers)
    df_cl = df_cl[df_cl["log2FoldChange"] > 0]
    # Sort by score (t-statistic) descending - highest = strongest marker
    df_cl = df_cl.sort_values("score", ascending=False).head(15).
 ↪reset_index(drop=True)
    tables.append(df_cl[["gene", "log2FoldChange", "pval", "pval_adj",␣
 ↪"score"]])

# Side-by-side view with cluster labels as top-level columns
de_top3 = pd.concat(tables, axis=1, keys=[f"cluster_{cl}" for cl in␣
 ↪clusters_to_show])

# Select columns by 2nd level of MultiIndex
lfc_cols = de_top3.columns[de_top3.columns.get_level_values(1) ==␣
 ↪"log2FoldChange"]
p_cols = de_top3.columns[de_top3.columns.get_level_values(1).isin(["pval",␣
 ↪"pval_adj"])]

de_top3.style.format("{:.3f}", subset=pd.IndexSlice[:, lfc_cols]) \
             .format("{:.2e}", subset=pd.IndexSlice[:, p_cols])
```

[20]:
```
<pandas.io.formats.style.Styler at 0x31cadb620>
```

These tables recapitualtes the DE assay results for the first 3 clusters, showing the top 15 genes for each cluster, along with their associated statistics such as log fold change, p-value, score of the t-test statistic. From this analysis, we can identify candidate marker genes that are highly expressed in specific clusters, which can be used to infer the potential cell types present in the dataset by comparing them with known markers from FlyBase and the scientific literature.

[21]:
```
cl, min_pct = "0", 0.10
lfc_thr, padj_thr = 1.0, 0.05

de = sc.get.rank_genes_groups_df(adata, group=cl).copy()
if {"pct_nz_group", "pct_nz_reference"}.issubset(de.columns):
    de = de[(de.pct_nz_group >= min_pct) | (de.pct_nz_reference >= min_pct)]
```

```
volcano = de.rename(columns={"names": "gene", "logfoldchanges": "log2FC",␣
↪"pvals_adj": "pval_adj"})[
    ["gene", "log2FC", "pval_adj"]
].astype({"log2FC": float, "pval_adj": float})

volcano = volcano[np.isfinite(volcano.log2FC) & np.isfinite(volcano.pval_adj) &␣
↪volcano.log2FC.between(-10, 10)].copy()
volcano["neglog10_padj"] = -np.log10(volcano.pval_adj.clip(lower=1e-300))
volcano["significant"] = (volcano.neglog10_padj > -np.log10(padj_thr)) &␣
↪(volcano.log2FC.abs() >= lfc_thr)

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(volcano.loc[~volcano.significant, "log2FC"], volcano.loc[~volcano.
↪significant, "neglog10_padj"], s=12, c="lightgray", alpha=0.7, linewidth=0)
ax.scatter(volcano.loc[volcano.significant, "log2FC"],  volcano.loc[volcano.
↪significant, "neglog10_padj"],  s=16, c="crimson",   alpha=0.85, linewidth=0)

ax.axvline(lfc_thr, color="black", ls="--", lw=1)
ax.axvline(-lfc_thr, color="black", ls="--", lw=1)
ax.axhline(-np.log10(padj_thr), color="black", ls="--", lw=1)

top = volcano[volcano.significant].nlargest(15, "neglog10_padj")
texts = [ax.text(r.log2FC, r.neglog10_padj, r.gene, fontsize=8) for r in top.
↪itertuples(index=False)]
adjust_text(texts, ax=ax, arrowprops=dict(arrowstyle="-", color="gray", lw=0.5))

ax.set(title=f"Volcano plot: cluster {cl} vs rest", xlabel="log2 fold change",␣
↪ylabel="-log10(adj p-value)")
plt.tight_layout()
plt.show()
```



The volcano plot above visualizes the differential expression results for cluster 0 compared to all other cells. Each point represents a gene, with its log2 fold change on the x-axis and the negative log10 of the adjusted p-value on the y-axis. Genes that are significantly upregulated in cluster 0 (meeting both fold change and p-value thresholds) are highlighted in red, while non-significant genes are shown in light gray. The dashed lines indicate the thresholds for significance. The top significant genes are annotated with their names for easy identification of potential markers for this cluster.

[22]:
```
de = sc.get.rank_genes_groups_df(adata, group=None)
min_pct, top_n = 0.10, 100

if {"pct_nz_group", "pct_nz_reference"}.issubset(de.columns):
    de = de[(de["pct_nz_group"] >= min_pct) | (de["pct_nz_reference"] >=␣
 ↪min_pct)]

de = (
    de.query("pvals_adj < 0.05 and logfoldchanges > 1.0")
      .assign(group_num=lambda d: pd.to_numeric(d["group"], errors="coerce"))
      .sort_values(["group_num", "group", "logfoldchanges"], ascending=[True,␣
 ↪True, False], na_position="last")
)
```

```
top100 = de.groupby("group", sort=False).head(top_n)
top100[["group", "names"]].to_csv("analysis/drscdb_upload_format.csv",␣
↪index=False, header=False)

de.groupby("group", sort=False).head(20).drop(columns="group_num").to_csv(
    "analysis/de_top20_per_cluster_readable.csv", index=False
)

print(f"Created upload file with {len(top100)} rows for DRscDB.")
print(f"Applied prevalence filter: pct_nz_group >= {min_pct:.0%} OR␣
↪pct_nz_reference >= {min_pct:.0%}")
```

```
Created upload file with 1796 rows for DRscDB.
Applied prevalence filter: pct_nz_group >= 10% OR pct_nz_reference >= 10%
```

The top 100 genes from each cluster will be then analyzed for their expression patterns across clusters, using first DRscDB, and then FlyBase to infer the likely cell type identity of each cluster. This must be done using the specific geneset of our sample provenance, which is the Drosophila ovary. - DRscDB is a database of single-cell RNA-seq data from Drosophila, which allows us to compare our identified marker genes with known expression profiles across various tissues and cell types in the fly. The database is provided by the Drosophila RNAi Screening Center (DRSC) at Harvard Medical School and can be acessed here. - FlyBase is a comprehensive database for Drosophila genetics and molecular biology, which includes extensive gene expression data and annotations. By comparing our marker genes with FlyBase, we can gain insights into the potential functions and tissue-specific expression patterns of these genes, further aiding in the annotation of cell types within our dataset. FlyBase can be accessed here.

8 6. Cell-type annotation

We can now fetch the results of our multiple enrichment analyis with DRscDB and FlyBase, and summarize them in a table that shows the top enriched gene sets for each cluster, along with their associated statistics such as fold change and adjusted p-value.

[23]:
```
enrich_df = pd.read_csv("analysis/enrichment.csv", index_col=1)
enrich_df.index.name = 'gene_set'

results = []
n_clusters = 18

for i in range(n_clusters):
    pval_col = f"P Value  {i}"
    fold_col = f"Fold Change  {i}"

    if pval_col in enrich_df.columns:
        # Convert to numeric
        temp = enrich_df[[pval_col, fold_col]].copy()
        temp[pval_col] = pd.to_numeric(temp[pval_col], errors='coerce')
```

```
        temp[fold_col] = pd.to_numeric(temp[fold_col], errors='coerce')

        # Get top 3 by smallest p-value and reset index
        top3 = temp.nsmallest(3, pval_col).reset_index()
        top3['cluster'] = i
        top3 = top3[['cluster', 'gene_set', pval_col, fold_col]]

        # Standardize column names BEFORE appending
        top3.columns = ['cluster', 'gene_set', 'p_value', 'fold_change']
        results.append(top3)

summary = pd.concat(results, ignore_index=True)
summary = summary.set_index(['cluster', 'gene_set'])
summary = summary.sort_values(['cluster', 'fold_change'], ascending=[True,␣
 ↪False])
summary.head(15)
```

[23]:

```
                                                          p_value  \
cluster gene_set
0       main body follicle cell ca. until St. 5         1.000000e-25
        follicle stem cell and prefollicle cell         1.000000e-25
        central main body follicle cell ca. St. 6-8     1.000000e-25
1       dorsal appendage forming follicle cell          1.000000e-25
        choriogenic main body follicle cell St. 14      1.000000e-25
        choriogenic main body follicle cell and corpus … 1.320000e-25
2       young germ cell                                 1.000000e-25
        artefact                                        1.000000e-25
        Germline 2                                      1.000000e-25
3       dorsal appendage forming follicle cell          1.000000e-25
        choriogenic main body follicle cell St. 12      1.000000e-25
        Dorsal Appendage Forming Follicle               1.000000e-25
4       escort cell                                     1.000000e-25
        3.2.0 stalk cell                                1.000000e-25
        3.0.1 FSC                                       1.000000e-25

                                                        fold_change
cluster gene_set
0       main body follicle cell ca. until St. 5             55.1515
        follicle stem cell and prefollicle cell             44.9920
        central main body follicle cell ca. St. 6-8         30.4784
1       dorsal appendage forming follicle cell              50.1588
        choriogenic main body follicle cell St. 14          34.8325
        choriogenic main body follicle cell and corpus …   29.2593
2       young germ cell                                     45.9789
        artefact                                            37.6191
        Germline 2                                          34.8325
3       dorsal appendage forming follicle cell              64.0918
```

```
        choriogenic main body follicle cell St. 12          44.5856
        Dorsal Appendage Forming Follicle                   34.8325
4       escort cell                                         62.6985
        3.2.0 stalk cell                                    32.0459
        3.0.1 FSC                                           30.6526
```

8.0.1 Cluster-to-marker evidence table

Cluster	Proposed cell type	DE marker genes (top 5)	DRscDB gene set support	Fold change
0	Main body follicle cell	CadN, Octalpha2R, fru, tutl, DIP-epsilon	main body follicle cell ca. until St. 5	55.15
1	Dorsal appendage follicle cell	Sarm, CG1273, tok, CG3168, CG42590	dorsal appendage forming follicle cell	50.16
2	Young germline cell	Fkbp12, fs(1)Ya, swa, gnu, snRNA:U11	young germ cell	45.98
3	Dorsal appendage follicle cell	if, srp, sick, CG10163, Cyp4p3	dorsal appendage forming follicle cell	64.09
4	Escort cell	robo2, side-VIII, sick, CG45263, CG1136	escort cell	62.70
5	Germline cyst (16-cell)	CG42260, CG2861, CG15628, X11Lbeta, mAChR-A	16-cell germline cyst in germarium	54.34
6	Differentiated germline cell	beat-Ic, solo, CG40006, Marf1, lncRNA:CR46148	Germline 2 / germ cell stage 4+	41.80
7	Oviduct cell	pio, Tep4, Tsp29Fb, Tsp42Ed, hth	oviduct	75.24
8	Choriogenic follicle cell (St. 14)	Cp18, ec, lncRNA:CR44472, Cp19, hebe	choriogenic main body follicle cell St. 14	105.89
9	Choriogenic follicle/CL cell	lncRNA:CR45453, lncRNA:CR43250, lncRNA:CR45756, Ilp8, Jhe	choriogenic main body follicle cell & corpus luteum	71.06
10	Posterior terminal follicle cell	ana, CG42747, pnt, Cad99C, H15, mid	posterior terminal follicle cell ca. St. 5-8	55.73

Cluster	Proposed cell type	DE marker genes (top 5)	DRscDB gene set support	Fold change
11	Young germline cell	RNASEK, Tom7, swa, yl, Tim8	young germ cell	73.84
12	Choriogenic follicle cell (St. 12)	CG31886, CG4661, CG6836, CG7888, CG7912	choriogenic main body follicle cell St. 12	94.74
13	Choriogenic follicle/CL cell	dpr8, lncRNA:CR45453, lectin-46Ca, Ilp8, slow	choriogenic MBFC & corpus luteum	58.52
14	Choriogenic follicle/CL cell	Paip2, CG9864, Ilp7, CG9626, nahoda	choriogenic main body follicle cell & corpus luteum	69.67
15	Choriogenic follicle cell (St. 10B)	Fcp3C, CG14834, Gale, ELOVL, CG14309	Choriogenic MBFCs (Stg. 10B)	51.55
16	Stretch follicle cell	cv-2, Cln3, CG7512, dpp, ImpL2	stretch follicle cell	89.17
17	Oviduct cell	CG9701, Cyp4d8, Dh44-R2, sNPF-R, Tsp42Ei	oviduct	64.09

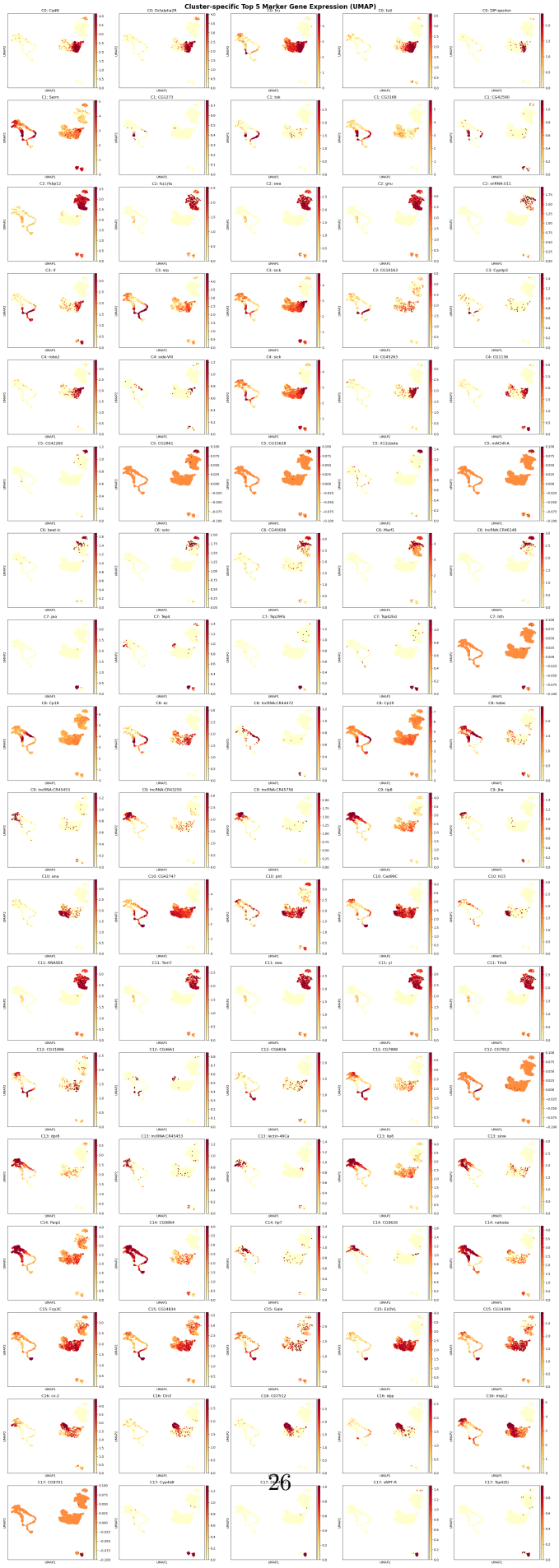
8.0.2 Remarks

- All clusters show large enrichment (p-values 1.32e-25, fold changes 15–106)
- Follicle cells are the dominant population (clusters 0, 1, 3, 8, 9, 10, 12, 13, 14, 15, 16) with stage-specific specialization
- Germline cells form a distinct lineage (clusters 2, 5, 6, 11)
- Oviduct cells (clusters 7, 17) represent a differentiated somatic tissue
- Escort cells (cluster 4) mark the somatic niche

[24]:
```
# Define marker genes per cluster from the table
marker_genes_per_cluster = {
    0: ['CadN', 'Octalpha2R', 'fru', 'tutl', 'DIP-epsilon'],
    1: ['Sarm', 'CG1273', 'tok', 'CG3168', 'CG42590'],
    2: ['Fkbp12', 'fs(1)Ya', 'swa', 'gnu', 'snRNA:U11'],
    3: ['if', 'srp', 'sick', 'CG10163', 'Cyp4p3'],
    4: ['robo2', 'side-VIII', 'sick', 'CG45263', 'CG1136'],
    5: ['CG42260', 'CG2861', 'CG15628', 'X11Lbeta', 'mAChR-A'],
    6: ['beat-Ic', 'solo', 'CG40006', 'Marf1', 'lncRNA:CR46148'],
    7: ['pio', 'Tep4', 'Tsp29Fb', 'Tsp42Ed', 'hth'],
    8: ['Cp18', 'ec', 'lncRNA:CR44472', 'Cp19', 'hebe'],
    9: ['lncRNA:CR45453', 'lncRNA:CR43250', 'lncRNA:CR45756', 'Ilp8', 'Jhe'],
```

```
    10: ['ana', 'CG42747', 'pnt', 'Cad99C', 'H15'],
    11: ['RNASEK', 'Tom7', 'swa', 'yl', 'Tim8'],
    12: ['CG31886', 'CG4661', 'CG6836', 'CG7888', 'CG7912'],
    13: ['dpr8', 'lncRNA:CR45453', 'lectin-46Ca', 'Ilp8', 'slow'],
    14: ['Paip2', 'CG9864', 'Ilp7', 'CG9626', 'nahoda'],
    15: ['Fcp3C', 'CG14834', 'Gale', 'ELOVL', 'CG14309'],
    16: ['cv-2', 'Cln3', 'CG7512', 'dpp', 'ImpL2'],
    17: ['CG9701', 'Cyp4d8', 'Dh44-R2', 'sNPF-R', 'Tsp42Ei'],
}
```

[25]:
```python
# Create a large grid: 18 clusters x 5 genes per cluster
fig, axes = plt.subplots(18, 5, figsize=(25, 70))
axes = axes.flatten()

ax_idx = 0
for cluster in range(18):
    genes = marker_genes_per_cluster[cluster]

    for gene in genes:
        ax = axes[ax_idx]

        # Check if gene exists in adata
        if gene in adata.var_names:
            sc.pl.umap(
                adata,
                color=gene,
                ax=ax,
                show=False,
                title=f'C{cluster}: {gene}',
                color_map='YlOrRd',
                vmax='p99',
                size=30
            )
        else:
            # Gene not found - show placeholder
            ax.text(0.5, 0.5, f'{gene}\nNOT FOUND',
                   ha='center', va='center', fontsize=10, color='red',
                   transform=ax.transAxes)
            ax.set_title(f'C{cluster}: {gene}')
            ax.axis('off')

        ax_idx += 1

plt.suptitle('Cluster-specific Top 5 Marker Gene Expression (UMAP)',
             fontsize=20, fontweight='bold', y=0.995)
plt.tight_layout()
plt.show()
```


Cluster-specific Top 5 Marker Gene Expression (UMAP)
C0: Cadfd
C0: Ostalpha2ff
C0: Ccl
C0: Ta0
C0: DIP-epsilon
C1: Serm
C1: CG1273
C1: Sok
C1: CG3168
C1: CG42590
C2: Fkbp12
C2: fs(1)Yb
C2: swa
C2: grxu
C2: snRNA:U11
C3: if
C3: srp
C3: sick
C3: CG30163
C3: Cyp4p3
C4: robo2
C4: side-VII
C4: sick
C4: CG45263
C4: CG1136
C5: CG42280
C5: CG2861
C5: CG15428
C5: X31beta
C5: mAChR-A
C6: beat-Ic
C6: solo
C6: CG40006
C6: Marf1
C6: lncRNA:CR46148
C7: jaa
C7: Tep4
C7: Tsp29Fb
C7: Tsp42Ed
C7: Ndn
C8: Cp18
C8: ec
C8: lncRNA:CR44472
C8: Cp19
C8: bebe
C9: lncRNA:CR45453
C9: lncRNA:CR43250
C9: lncRNA:CR45756
C9: Ilp8
C9: jhe
C10: ana
C10: CG42747
C10: ptt
C10: Cad96C
C10: H15
C11: RNASEK
C11: Tom7
C11: swa
C11: yl
C11: Timd
C12: CG13896
C12: CG4961
C12: CG4834
C12: CG7880
C12: CG7912
C13: dpr8
C13: lncRNA:CR45453
C13: lectin-46Ca
C13: Ilp8
C13: slow
C14: Pelp2
C14: CG9064
C14: Itp7
C14: CG8626
C14: nahoda
C15: Fcp3C
C15: CG14834
C15: Gale
C15: EcDV1
C15: CG14309
C16: ck-2
C16: Ctn3
C16: CG7512
C16: dpp
C16: knaz.2
C17: CG8781
C17: Cyp4d8
C17: sNPF-R
C17: Tsp42Ef
UMAP1
UMAP2

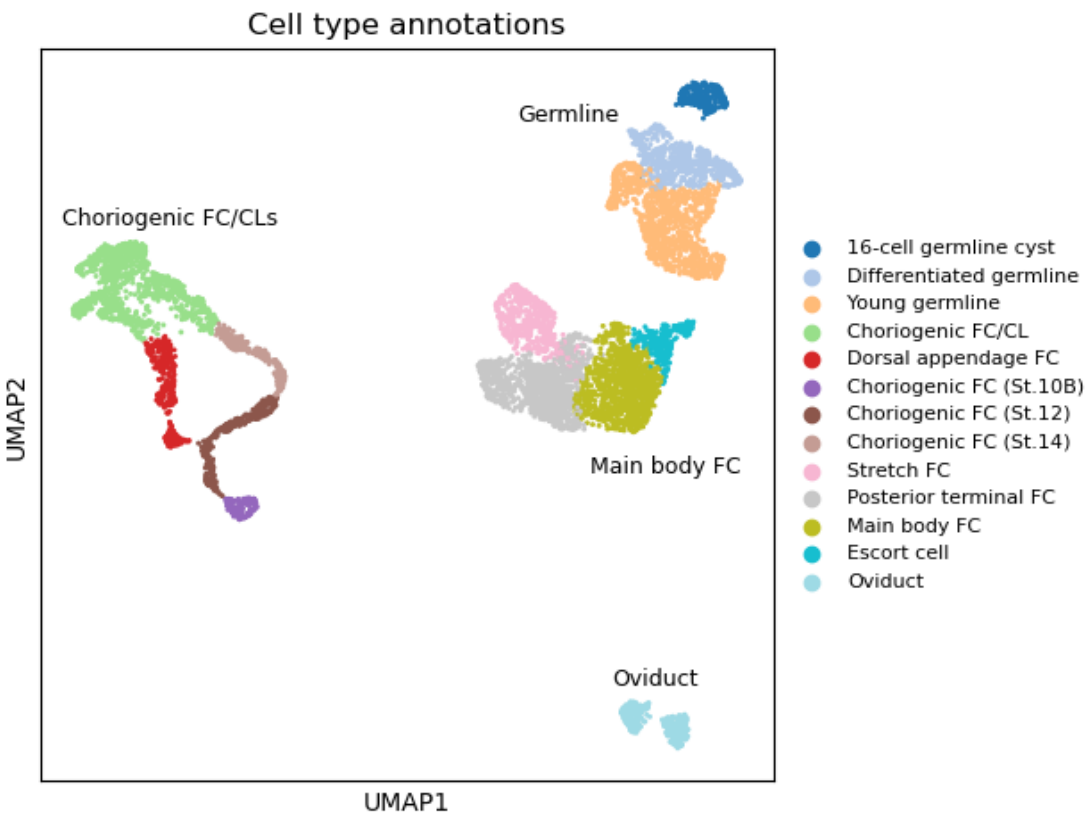
On this gigantic figure, each row is a cluster displaying its 5 top marker genes on the UMAP, to colocalize their expression patterns. We observe that the top marker genes for each cluster are highly expressed in specific regions of the UMAP, which correspond to the identified clusters. Some genes can be also expressed in multiple clusters, indicating shared expression patterns, or even expresse throughout the entire dataset, which may suggest housekeeping functions or broad expression across cell types.

Indeed, to assign biological identities to the clusters observed in the UMAP embedding, we examined the expression of an **extended panel of known marker genes** associated with Drosophila ovary cell types. By comparing the spatial expression patterns of these markers with the Leiden clusters, we can manually annotate each cluster with a **putative cell type identity**.

Based on those results, we make the cluster assignments :

[26]:
```
# Visually inspecting the plots above leads to assigning cell type labels
cell_type_labels = {
    "0": "Main body FC", "1": "Dorsal appendage FC", "2": "Young germline", "3":
 ↪ "Dorsal appendage FC",
    "4": "Escort cell", "5": "16-cell germline cyst", "6": "Differentiated␣
 ↪germline", "7": "Oviduct",
    "8": "Choriogenic FC (St.14)", "9": "Choriogenic FC/CL", "10": "Posterior␣
 ↪terminal FC",
    "11": "Young germline", "12": "Choriogenic FC (St.12)", "13": "Choriogenic␣
 ↪FC/CL",
    "14": "Choriogenic FC/CL", "15": "Choriogenic FC (St.10B)", "16": "Stretch␣
 ↪FC", "17": "Oviduct",
}

legend_order = [
    "16-cell germline cyst", "Differentiated germline", "Young germline",
    "Choriogenic FC/CL", "Dorsal appendage FC", "Choriogenic FC (St.10B)",
    "Choriogenic FC (St.12)", "Choriogenic FC (St.14)", "Stretch FC",
    "Posterior terminal FC", "Main body FC", "Escort cell", "Oviduct",
]

mapped = adata.obs["leiden"].map(cell_type_labels)
present = [c for c in legend_order if c in pd.unique(mapped.dropna())]
adata.obs["cell_type"] = pd.Categorical(mapped, categories=present,␣
 ↪ordered=True)

sc.pl.umap(
    adata, color="cell_type", legend_loc="right margin", legend_fontsize=8,
    title="Cell type annotations", palette="tab20", show=False
)
```

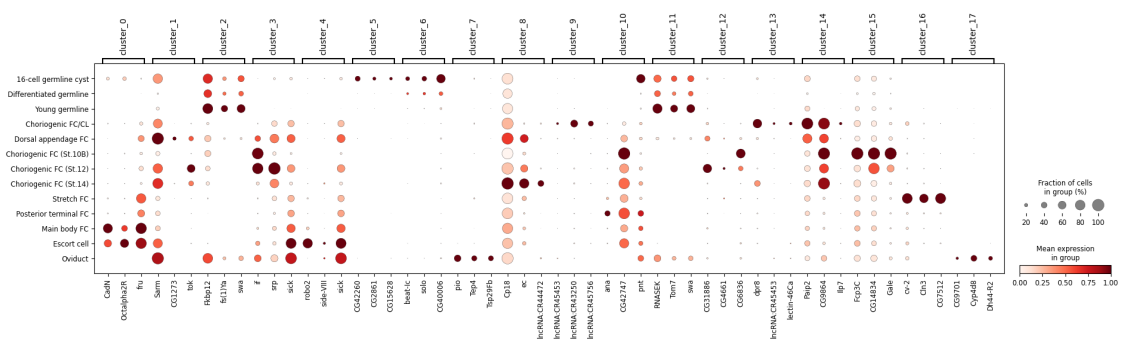
```
plt.annotate('Choriogenic FC/CLs', xy=(0.03, 0.76), xycoords='axes fraction',␣
↪fontsize=9, color='black')
plt.annotate('Main body FC', xy=(0.75, 0.42), xycoords='axes fraction',␣
↪fontsize=9, color='black')
plt.annotate('Germline', xy=(0.65, 0.9), xycoords='axes fraction', fontsize=9,␣
↪color='black')
plt.annotate('Oviduct', xy=(0.78, 0.13), xycoords='axes fraction', fontsize=9,␣
↪color='black')

plt.tight_layout()
plt.savefig('results/cell_types_umap.png', dpi=300, bbox_inches='tight')
plt.show()
```



The annotated cell types visualized on the UMAP plot highlight the spatial organization of these populations and confirme the presence of several biologically meaningful cell groups within the dataset.

[27]:
```
gene_index = adata.raw.var_names if adata.raw is not None else adata.var_names

marker_genes_for_plot = {
    f"cluster_{cl}": [g for g in genes if g in gene_index][:3]
```

```
    for cl, genes in marker_genes_per_cluster.items()
}

marker_genes_for_plot = {k: v for k, v in marker_genes_for_plot.items() if␣
↪len(v) > 0}

sc.pl.dotplot(
    adata,
    var_names=marker_genes_for_plot,
    groupby="cell_type",
    standard_scale="var",
    cmap="Reds",
    use_raw=(adata.raw is not None),
    show=False,
)

# plt.title("Expression of Top Marker Genes Across Cell Types", fontsize=14,␣
↪fontweight='bold')
plt.savefig('results/marker_genes_dotplot.png', dpi=300, bbox_inches='tight')
plt.show()
```



9 Final Answers

9.0.1 Q1. How many cell types are in the dataset?

- Number of Leiden clusters detected: **18 (clusters 0–17)**
- Number of validated biological cell types: **13**

This is due to the fact that some clusters correspond to the same cell type (for example, clusters 1 and 3 both correspond to dorsal appendage follicle cells, and clusters 7 and 17 both correspond to oviduct cells). While they still represent distinct transcriptional states, they are likely subpopulations of the same broader cell type.

9.0.2 Q2. Which marker genes characterize the cell types?

Cell type (validated)	Main cluster(s)	Marker genes used
Main body FC	0	CadN, Octalpha2R, fru, tutl, DIP-epsilon
Dorsal appendage FC	1, 3	Sarm, CG1273, tok, CG3168, CG42590; if, srp, sick, CG10163, Cyp4p3
Young germline	2, 11	Fkbp12, fs(1)Ya, swa, gnu, snRNA:U11; RNASEK, Tom7, yl, Tim8
Escort cell	4	robo2, side-VIII, sick, CG45263, CG1136
16-cell germline cyst	5	CG42260, CG2861, CG15628, X11Lbeta, mAChR-A
Differentiated germline	6	beat-Ic, solo, CG40006, Marf1, lncRNA:CR46148
Oviduct	7, 17	pio, Tep4, Tsp29Fb, Tsp42Ed, hth; CG9701, Cyp4d8, Dh44-R2, sNPF-R, Tsp42Ei
Choriogenic FC (St.14)	8	Cp18, ec, lncRNA:CR44472, Cp19, hebe
Choriogenic FC/CL	9, 13, 14	lncRNA:CR45453, lncRNA:CR43250, lncRNA:CR45756, Ilp8, Jhe; dpr8, lectin-46Ca, slow; Paip2, CG9864, Ilp7, CG9626, nahoda
Posterior terminal FC	10	ana, CG42747, pnt, Cad99C, H15
Choriogenic FC (St.12)	12	CG31886, CG4661, CG6836, CG7888, CG7912
Choriogenic FC (St.10B)	15	Fcp3C, CG14834, Gale, ELOVL, CG14309
Stretch FC	16	cv-2, Cln3, CG7512, dpp, ImpL2

9.0.3 Q3. What cell types are likely present?

Broad type	Validated cell types
Germline lineage	16-cell germline cyst; Young germline; Differentiated germline
Follicle-cell lineage	Main body FC; Dorsal appendage FC; Posterior terminal FC; Stretch FC; Choriogenic FC (St.10B); Choriogenic FC (St.12); Choriogenic FC (St.14); Choriogenic FC/CL
Other somatic cells	Escort cell; Oviduct

- The dataset is dominated by follicle-cell lineages, especially multiple stage-specific choriogenic follicle subtypes, alongside main body, dorsal appendage, posterior terminal, and stretch follicle populations.
- A clear germline axis is present (young germline, differentiated germline, and 16-cell cyst states).
- Additional distinct somatic populations include escort cells and oviduct cells, which are well-separated in UMAP and consistently supported by DRscDB enrichment.